

Todo App Study Notes

Todo App Study Notes (Simple + Organized)

These notes explain the core ideas used in a Kotlin + Jetpack Compose Todo app.

The goal is to understand what each part does, why we use it, and the flow from DB to UI.

1) MainViewModelFactory (How ViewModel gets dependencies)

```
class MainViewModelFactory(  
    private val repository: TodoRepository  
) : ViewModelProvider.Factory {  
    override fun <T : ViewModel> create(modelClass: Class<T>): T {  
        if (modelClass.isAssignableFrom(MainViewModel::class.java)) {  
            @Suppress("UNCHECKED_CAST")  
            return MainViewModel(repository) as T  
        }  
        throw IllegalArgumentException("Unknown ViewModel class")  
    }  
}
```

What it does

- Creates MainViewModel manually.
- Passes TodoRepository into ViewModel constructor.

Why use it

- Default viewModel() cannot pass custom constructor parameters directly.
- Factory solves dependency injection in simple apps.

Flow

1. UI asks for ViewModel.
 2. Factory create() is called.
 3. If requested class is MainViewModel, return it with repository.
 4. If not, throw error.
-

2) Repository + Flow mapping (observeTodos)

```
fun observeTodos(): Flow<List<TodoItem>> {  
    return dao.observeTodos().map { entities ->  
        entities.map { it.toUi() }  
    }  
}
```

What it does

- Reads DB stream (Flow<List<TodoEntity>>) from DAO.
- Converts DB model (TodoEntity) to UI/domain model (TodoItem).
- Returns Flow<List<TodoItem>>.

Step-by-step flow

1. dao.observeTodos() emits whenever table changes (insert/update/delete).
2. Outer map { ... } is Flow.map (each emission is transformed).
3. Inner entities.map { ... } is List.map (each row transformed).
4. Output becomes Flow<List<TodoItem>>.
5. UI collects and updates automatically.

Simple meaning

Whenever DB changes, convert rows to app-friendly objects and send fresh list to UI.

3) Mapping utility methods (Entity <-> UI model)

```
private fun TodoEntity.toUi() = TodoItem(
    id = id,
    title = title,
    description = description,
    createdOn = createdOn,
    priority = priority
)

private fun TodoItem.toEntity() = TodoEntity(
    id = id,
    title = title,
    description = description,
    createdOn = createdOn,
    priority = priority
)
```

Why this is useful

- Keeps conversion logic in one place.
- Cleaner repository/DAO code.
- Easy to maintain if model changes later.

4) ViewModel state with StateFlow

```
private val _todos = MutableStateFlow<List<TodoItem>>(emptyList())
val todos: StateFlow<List<TodoItem>> = _todos.asStateFlow()

init {
```

```
viewModelScope.launch {
    repo.observeTodos().collect { list ->
        _todos.value = list
    }
}
```

What it does

- _todos is mutable inside ViewModel.
- todos is read-only outside ViewModel.
- ViewModel collects repository Flow and updates state.

Why use this pattern

- Encapsulation: only ViewModel can mutate state.
- UI gets safe read-only stream.
- Works perfectly with Compose.

5) Compose observes ViewModel (collectAsState)

```
val todos by vm.todos.collectAsState()
```

What it does

- vm.todos is StateFlow<List<TodoItem>>.
- collectAsState() converts Flow into Compose State.
- by gives direct value (todos) instead of state.value.

Result

- UI automatically recomposes when todos change.

6) LazyColumn item key (id) to reduce recomposition issues

```
items(todos, key = { it.id }) { todo ->
    TodoItemComposable(todoItem = todo)
}
```

Why key matters

- Compose can track each item identity using stable id.
- Better list performance.
- Prevents wrong item state reuse when list order changes.
- Helps minimize unnecessary recompositions/rebind behavior.

7) FAB (Floating Action Button) event flow

```
floatingActionButton = {
    FloatingActionButton(
        onClick = {
            vm.onAddTodo(
                title = "New Todo",
                description = "Created from FAB",
                priority = Priority.MEDIUM
            )
        }
    ) {
        Icon(Icons.Default.Add, contentDescription = "Add Todo")
    }
}
```

Flow

1. User taps FAB.
2. vm.onAddTodo(...) is called.
3. ViewModel inserts into DB (through repository).
4. Room Flow emits updated list.
5. UI gets new state and redraws list.

8) Spacer usage (Spacer(Modifier.weight(1f)))

```
Spacer(Modifier.weight(1f))
```

What it does

- Takes remaining free space in a Row/Column.
- Pushes other components apart.

Simple use case

- Push button to bottom/end.
- Create flexible layout spacing without hardcoded dp.

9) Empty state UI pattern

```
if (todos.isEmpty()) {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text("No todo")
    }
} else {
```

```

LazyColumn {
    items(todos, key = { it.id }) { todo ->
        TodoItemComposable(todoItem = todo)
    }
}
}

```

Why important

- Better UX: user knows list is empty, not broken/loading forever.
- Clear state handling in UI.

10) Firestore setup (Gradle)

Project build.gradle.kts

```

plugins {
    // existing...
    id("com.google.gms.google-services") version "4.4.2" apply false
}

```

App build.gradle.kts

```

plugins {
    // existing...
    id("com.google.gms.google-services")
}

dependencies {
    // existing...
    implementation(platform("com.google.firebase:firebase-bom:33.12.0"))
    implementation("com.google.firebase:firebase-firestore-ktx")
}

```

Why use BOM

- Firebase BOM manages compatible versions across Firebase libraries.

11) Firestore CRUD sample

```

class FirestoreTodoDataSource {
    private val db = FirebaseFirestore.getInstance()
    private val todosRef = db.collection("todos")

    fun pushTodo(todo: TodoItem, onResult: (Boolean, String?) -> Unit) {
        val map = hashMapOf(
            "id" to todo.id,

```

```

        "title" to todo.title,
        "description" to todo.description,
        "createdOn" to todo.createdOn,
        "priority" to todo.priority.name
    )
    todosRef.document(todo.id).set(map)
        .addOnSuccessListener { onResult(true, null) }
        .addOnFailureListener { e -> onResult(false, e.message) }
    }

fun deleteTodo(id: String, onResult: (Boolean, String?) -> Unit) {
    todosRef.document(id).delete()
        .addOnSuccessListener { onResult(true, null) }
        .addOnFailureListener { e -> onResult(false, e.message) }
}

fun observeTodos(onData: (List<TodoItem>) -> Unit): ListenerRegistration {
    return todosRef.addSnapshotListener { snapshot, _ ->
        val list = snapshot?.documents?.mapNotNull { doc ->
            val id = doc.getString("id") ?: return@mapNotNull null
            val title = doc.getString("title") ?: ""
            val description = doc.getString("description") ?: ""
            val createdOn = doc.getLong("createdOn") ?: 0L
            val priority = Priority.valueOf(doc.getString("priority") ?: "LOW")
            TodoItem(id, title, description, createdOn, priority)
        }.orEmpty()
        onData(list)
    }
}
}

```

Note

- Firestore callbacks here are listener-based (not Flow by default).
- You can wrap this in Flow later for same reactive style.

12) StateFlow + State Hoisting (important architecture idea)

UI state model

```

data class TodoUiState(
    val todos: List<TodoItem> = emptyList(),
    val isLoading: Boolean = false
)

```

ViewModel owns state

```

class MainViewModel(app: Application) : AndroidViewModel(app) {
    private val repo = TodoRepository(app)
    private val _uiState = MutableStateFlow(TodoUiState(isLoading = true))
    val uiState: StateFlow<TodoUiState> = _uiState.asStateFlow()

    init {
        viewModelScope.launch {
            repo.observeTodos().collect { list ->
                _uiState.value = TodoUiState(
                    todos = list,
                    isLoading = false
                )
            }
        }
    }
}

```

Route collects state, passes down

```

@Composable
fun TodoRoute(vm: MainViewModel = viewModel()) {
    val uiState by vm.uiState.collectAsState()
    TodoScreen(
        todos = uiState.todos,
        isLoading = uiState.isLoading,
        onAddClick = { /* vm.onAddTodo(...) */ }
    )
}

```

Stateless screen (state hoisting)

```

@Composable
fun TodoScreen(
    todos: List<TodoItem>,
    isLoading: Boolean,
    onAddClick: () -> Unit
) {
    when {
        isLoading -> Text("Loading...")
        todos.isEmpty() -> Text("No todo")
        else -> LazyColumn {
            items(todos, key = { it.id }) { todo ->
                TodoItemComposable(todoItem = todo)
            }
        }
    }
}

```

```
        FloatingActionButton(onClick = onAddClick) { Text("+") }  
    }
```

Teaching line

- TodoRoute is stateful container (talks to ViewModel).
- TodoScreen is stateless UI (renders data + sends events).
- This is state hoisting: state goes down, events go up.

13) suspend in simple words (code meaning)

In Kotlin coroutines, suspend means:

pause this function without blocking the thread, then continue later.

What it does

- Function can pause at suspension points (delay, network, DB).
- Thread is free during pause.
- Function resumes from same point later.

Why use it

- Keeps UI responsive.
- More efficient than blocking calls.
- Async code looks clean and sequential.

Step-by-step flow

1. Call suspend function inside coroutine.
2. Function starts.
3. Hits suspension point.
4. Pauses and returns thread to runtime.
5. Runtime uses thread for other tasks.
6. When result is ready, function resumes.
7. Continues from pause point and returns.

Tiny example

```
suspend fun loadUser(): User {  
    delay(1000)           // non-blocking pause  
    return api.getUser()  // maybe another suspend call  
}
```

Quick Full-App Mental Flow

1. User taps UI (like FAB).
2. ViewModel handles event.

3. Repository writes/reads DB.
4. Room Flow emits new list.
5. ViewModel updates StateFlow.
6. Compose collects state via collectAsState.
7. UI recomposes (with stable list keys via id).

This is the core reactive loop of your Todo app.